

# Tema 8.5: Fundamentos y Estrategias de Depuración (Debugging)

---

## Resumen

---

En esta lectura, usted estudiará los conceptos esenciales de la depuración de software (*debugging*), comprendiendo los procesos sistemáticos para localizar y resolver fallas lógicas y de ejecución. Aprenderá a configurar y operar sesiones de depuración dinámica en **Visual Studio Code** para código Java y entornos web, y a integrar herramientas de inteligencia artificial como **Google Antigravity** para diagnosticar excepciones complejas y trazas de error de manera ágil y estructurada.

---

## Concepto Principal (El 'Qué')

---

La **depuración (*debugging*)** es el proceso sistemático e incremental de identificar, aislar y corregir defectos (*bugs*) en el código fuente de una aplicación. A diferencia de las pruebas de software, que se limitan a constatar la presencia de un error bajo ciertas condiciones, la depuración busca comprender la causa raíz del fallo y la inconsistencia entre el comportamiento esperado y el real.

Existen dos aproximaciones metodológicas para analizar el código:

- **Análisis Estático:** Consiste en examinar el código sin ejecutarlo. Se realiza mediante herramientas como linters o el compilador del IDE, que resaltan errores sintácticos, variables sin uso o malas prácticas.
- **Análisis Dinámico:** Consiste en observar el comportamiento del sistema en tiempo de ejecución. Aquí es donde interviene el depurador (*debugger*), que permite detener e inspeccionar el estado exacto de la memoria.

## Conceptos Clave de la Depuración Dinámica

Para operar de forma profesional un depurador, se deben dominar los siguientes elementos:

- **Punto de Interrupción (*Breakpoint*):** Marca colocada en una línea de código específica que le indica al entorno de ejecución que suspenda temporalmente el programa justo antes de ejecutar esa instrucción. Permite congelar el tiempo del sistema.
- **Pila de Llamadas (*Call Stack*):** Historial ordenado de los métodos activos que se han ido invocando hasta llegar a la línea actual. Permite entender el camino que recorrió el flujo de ejecución.
- **Inspección de Variables y Watch:** Panel de visualización donde el desarrollador puede ver el valor en tiempo real de las variables locales presentes en el contexto actual, o definir expresiones específicas a monitorear.
- **Comandos de Control de Flujo:**
  - **Step Over (F10):** Ejecuta la línea actual de código y se detiene en la siguiente línea del mismo método, tratando las llamadas a métodos internos como una única instrucción rápida.
  - **Step Into (F11):** Si la línea actual posee una llamada a otro método de su propio código, el depurador ingresa a dicho método para inspeccionar su comportamiento interno línea por línea.
  - **Step Out (Shift + F11):** Ejecuta de forma rápida el resto del método actual y detiene el flujo en la línea inmediata posterior a la que invocó dicho método en la pila de llamadas.

---

## Analogía (La Intuición)

---

Depurar código es equivalente a ser un mecánico de precisión diagnosticando una falla en una línea de ensamblaje automatizada de vehículos.

Si un sensor detecta que las puertas de un auto están saliendo mal alineadas (falla visible), el mecánico no derriba la fábrica ni adivina el error tirando herramientas al azar. En su lugar, coloca "sensores de parada" (puntos de interrupción) en secciones específicas de la cinta de ensamblaje.

Cuando el robot avanza, la cinta se detiene exactamente en el robot de bisagras. El mecánico camina a la máquina, mide la presión y el ángulo físico en ese preciso instante (inspección de variables), avanza un paso controlado del brazo mecánico (Step Over) y observa si el movimiento es el correcto hasta descubrir que el calibrador del robot tiene un valor erróneo.

---

## Ejemplo de Código e Implementación (El 'Cómo')

A continuación, usted creará una configuración de depuración en Visual Studio Code para un proyecto Java y examinará un escenario de error real.

### Paso 1: Configuración de la Sesión en VS Code (`launch.json`)

Para que Visual Studio Code sepa cómo enlazar el depurador con sus aplicaciones, configure el archivo de lanzamiento. En el directorio raíz del proyecto, cree una carpeta llamada `.vscode/` y dentro de ella el archivo `launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "java",
      "name": "Debug Java Server",
      "request": "launch",
      "mainClass": "cr.ac.ucr.paraíso.ie.carnet.debug.AppServer"
    },
    {
      "type": "chrome",
      "name": "Debug Frontend in Chrome",
      "request": "launch",
      "url": "http://localhost:8080",
      "webRoot": "${workspaceFolder}/public"
    }
  ]
}
```

### Paso 2: Escenario de Error en Java (`NullPointerException`)

Cree la clase `CalculadoraSalarios.java` en la ruta `src/main/java/cr/ac/ucr/paraíso/ie/carnet/debug/CalculadoraSalarios.java`. Esta clase posee un error común que causa un fallo de ejecución en tiempo de producción:

```
package cr.ac.ucr.paraíso.ie.carnet.debug;

import java.util.ArrayList;
import java.util.List;
```

```

public class CalculadoraSalarios {
    public static void main(String[] args) {
        List<Empleado> empleados = obtenerEmpleados();
        double total = 0;

        for (Empleado emp : empleados) {
            // Error potencial: ¿Qué ocurre si un elemento es nulo?
            total += emp.getSalarioNeto();
        }

        System.out.println("Total salarios: " + total);
    }

    private static List<Empleado> obtenerEmpleados() {
        List<Empleado> lista = new ArrayList<>();
        lista.add(new Empleado("Ana", 1500));
        lista.add(null); // Elemento nulo intencional que causará la falla
        lista.add(new Empleado("Luis", 1200));
        return lista;
    }

    static class Empleado {
        String nombre;
        double salario;

        Empleado(String nombre, double salario) {
            this.nombre = nombre;
            this.salario = salario;
        }

        double getSalarioNeto() {
            return this.salario * 0.91; // Deducción de cargas sociales
        }
    }
}

```

### Flujo de Depuración Práctica:

1. Abra `CalculadoraSalarios.java` y coloque un punto de interrupción haciendo clic a la izquierda del número de línea en la línea `for (Empleado emp : empleados) {` (se mostrará un círculo rojo).
2. Presione la tecla **F5** para iniciar la depuración. La ejecución se detendrá en la línea marcada y esta se resaltará en amarillo.

3. Observe el panel lateral izquierdo en la sección **Variables**: podrá ver el objeto `empleados` y expandir su lista interna para inspeccionar sus 3 elementos, identificando que el índice `1` es `null`.
4. Presione **F10** (Step Over) para avanzar una línea. Al entrar a la primera iteración, la variable local `emp` tendrá los datos de "Ana".
5. Vuelva a presionar **F10**. En la segunda iteración, al intentar resolver `emp.getSalarioNeto()` siendo `emp` de tipo `null`, el sistema arrojará la excepción. Gracias a la pausa previa, usted pudo determinar con precisión matemática el elemento causante del fallo.

---

### Paso 3: Depuración Asistida con la IA (Google Antigravity)

En entornos de desarrollo profesionales, las trazas de error (*stack traces*) de frameworks como Spring Boot o Hibernate pueden ser extremadamente extensas y crípticas. Usted puede apoyarse en su asistente de IA integrado para resolver y corregir fallas mediante prompts efectivos:

- **Para explicar excepciones complejas:**  
*"Asistente, el compilador me genera el siguiente error en consola: [pegar excepción completa]. Explique de manera simple por qué ocurre este error y qué variables o flujos debería inspeccionar en mi código."*
- **Para solicitar un plan de reparación de código:**  
*"Asistente, analice el siguiente fragmento de código [pegar código] que lanza un `NullPointerException` y proponga una solución defensiva que prevenga el fallo sin alterar las reglas del negocio."*
- **Para analizar logs de red del navegador:**  
*"Asistente, mi frontend web está recibiendo un código de error `HTTP 415 Unsupported Media Type`. Explique qué discrepancia hay entre los encabezados del cliente y el servidor y cómo solucionarlo."*

---

## Trampas Comunes

---

- **No Recompilar en Cambios de Código:** Al ejecutar sesiones de depuración en Java nativo o Maven, si modifica el código fuente e inicia el depurador de inmediato sin realizar previamente la compilación (`mvn compile`), el depurador utilizará los archivos binarios antiguos. Esto causará un desfase en el IDE, donde el puntero de depuración amarillo apuntará a líneas de código vacías o erróneas.
- **Puntos de Interrupción no Enlazados (Círculo Gris):** Si al iniciar el depurador un breakpoint rojo se vuelve gris y muestra un mensaje indicando que no se pueden cargar los símbolos de depuración, esto suele deberse a que el compilador no generó la metadata de depuración (archivo `.class` compilado sin parámetros `-g` en `javac`). Asegúrese de limpiar la carpeta de salida (`mvn clean compile`) y reiniciar el depurador.
- **Ingresar a Código Externo del JDK:** Al presionar por error **Step Into (F11)** en una línea que invoca métodos de la librería estándar (como `System.out.println` o `List.add`), el depurador intentará abrir el código fuente interno de Java, lo cual confunde a los estudiantes principiantes. Si esto sucede, presione **Step Out (Shift + F11)** para regresar a su propio código.

---

## Pregunta de Reflexión

---

En el desarrollo de servicios web modernos basados en APIs REST:

¿Por qué el uso de herramientas como el depurador del navegador (DevTools - pestaña Network) es complementario y necesario a la depuración de consola en el Backend cuando se diagnostican errores de comunicación y parsing de JSON? ¿Qué problemas de formato se detectan en el cliente que nunca llegarían a ser visibles en los breakpoints del servidor Java?